Tab 1

API Test Automation Framework for E-Commerce using Karate

PROJECT REPORT

Submitted to

DEPARTMENT OF COMPUTER SCIENCE
GOBI ARTS & SCIENCE COLLEGE
(AUTONOMOUS)

GOBICHETTIPALAYAM - 638453.

By

DIVYA V

(24-06-103)

Guided By

G.PRABHU,M.Sc(CS).,M.Phil.,

In partial fulfillment of the requirements for the award of the degree of **Master of Science (Computer Science),** in the faculty of Computer Science in Gobi Arts & Science College (Autonomous), affiliated to Bharathiar University, Coimbatore.

APRIL-2026

DECLARATION

DECLARATION

I hereby declare that the project report entitled **"API Test Automation Framework for E-Commerce using Karate"** submitted to the Principal, **Gobi Arts & Science College (Autonomous), Gobichettipalayam**, in partial fulfillment of the requirements for the award of the degree of **Master of Science (Computer Science)**, is a record of project work done by me during the period of study in this college under the guidance and supervision of **G.PRABHU,M.Sc(CS).,M.Phil.,** Assistant Professor, Department of Computer Science.

SIGNATURE :

NAME : V. Divya

ROLL NO : 24-06-103

DATE :

CERTIFICATE

CERTIFICATE

This is to certify that the project report entitled **"API Test Automation Framework for E-Commerce using Karate "** is a bonafide work done by **DIVYA .V (24-06-103)** under my supervision and guidance.

Signature of the guide :

Name : G. PRABHU

Designation : Assistant professor

Department : Computer Science

Date :

Counter—Signed

Head of the Department **Principal**

Viva-voce held on: _____

Internal Examiner External **Examiner**

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

I would like to express my deep sense of gratitude and obligation to the college council for providing necessary facilities to give me the opportunity to do the entire college studies in Gobi Arts & Science College(Autonomous), Gobichettipalayam.

I wish to record my deep sense of gratitude to our beloved Principal **Dr.D.VENUGOPAL, M.Sc.,M.Phil.,PGDCA.,Ph.D.,** Gobi & Arts Science College(Autonomous) , Gobichettipalayam for his inspiration which made me to complete this project successfully.

I would like to acknowledge my gratitude to our beloved Head of the Department **Dr.K.A.SENTHILDEVI, M.C.A.,M.Phil.,Ph.D.,** AssociateProfessor, Department Of Computer Science, Gobi Arts & Science College(Autonomous),Gobichettipalayam cheerful encouraging and supporting me as a lot who have given me overwhelming support this project has been completed within the stipulated time.

I express my sincere thanks and gratitude to my project guide **G.PRABHU,M.Sc(CS)., M.Phil.,** Assistant Professor, Department of Computer Science, Gobi Arts & Science College (Autonomous), Gobichettipalayam with his valuable guidance and constructive suggestion, kind cooperation and encouragement throughout the project.

I am very much grateful to all faculty members of Computer Science and the programmers for their effort to complete this project successfully.

Finally 1 express my thanks to my family and my friends for their moral support to this project work.

V. DIVYA

SYNOPSIS

SYNOPSIS

This project entitled **"API Test Automation Framework for E-Commerce"** is a production-ready, scalable automation solution designed to validate the complex backend operations of modern online retail systems. The framework focuses on automating the complete lifecycle of e-commerce transactions, including user authentication, product catalog management, shopping cart operations, and order processing, ensuring reliable and error-free API functionality.

The system replaces traditional, code-intensive automation approaches with a streamlined **Behavior Driven Development (BDD)** methodology using the Karate Framework. By leveraging human-readable Gherkin syntax, the framework enables testers and developers to design test scenarios efficiently without complex programming. This approach improves maintainability, reduces development effort, and enhances collaboration between technical and non-technical stakeholders.

The framework is structured into specialized functional modules that reflect real-world business logic. These modules perform comprehensive validation of state transitions, business rules, and financial calculations, such as verifying pricing, discounts, and total amounts during checkout. A key feature of the system is its **data-driven architecture**, which separates test logic from external data sources like JSON and CSV files, allowing the same test cases to be executed with multiple datasets to ensure extensive coverage and reliability.

To further enhance system robustness, the framework integrates **performance testing and reporting capabilities** using Gatling to simulate high-concurrency traffic and evaluate API responsiveness under heavy loads. Built on a Java-based Maven environment and integrated with CI/CD tools such as Jenkins and GitHub Actions, the framework supports continuous testing and automated deployments. With JUnit orchestration and mock server integration, the project provides a complete ecosystem for functional, regression, and performance testing.

CONTENTS

CONTENTS

ACKNOWLEDGEMENT		i
SYNOPSIS		ii
CHAPTER	**TITLE**	**PAGE No.**
1.	**INTRODUCTION**	**1**
	1.1 ORGANIZATION PROFILE	1
	1.2 ABOUT THE PROJECT	2
	1.3 HARDWARE SPECIFICATION	3
	1.4 SOFTWARE SPECIFICATION	4
2.	**SYSTEM ANALYSIS**	**7**
	2.1 PROBLEM DEFINITION	7
	2.2 SYSTEM STUDY	8
	2.3 PROPOSED SYSTEM	9
3.	**SYSTEM DESIGN**	**11**
	3.1 DATA FLOW DIAGRAM	11
	3.2 E-R DIAGRAM	13
	3.3 FILE SPECIFICATION	14
	3.4 MODULE SPECIFICATION	20
4.	**TESTING AND IMPLEMENTATION**	**27**
5.	**CONCLUSION AND SUGGESTIONS**	**28**
	BIBLIOGRAPHY	**29**

APPENDICES

PAGE No.

APPENDIX - A **30**

A1. ADMIN LOGIN PAGE 30

A2. ADMIN HOME PAGE 31

A3. GROUP ENTRY DETAILS 32

A4. USER ENTRY DETAILS 33

A5. CHAT ENTRY DETAILS 34

A6. FEEDBACK ENTRY DETAILS 35

A7. DOCTOR ENTRY DETAILS 36

A8. PATIENT ENTRY DETAILS 37

APPENDIX - B (REPORTS) **38**

B1. GROUP REPORT 38

B2. USER REPORT 39

B3. CHAT REPORT 40

B4. FEEDBACK REPORT 41

B5. DOCTOR INVOICE 42

B6. PATIENT REPORT 43

B6. RATING REPORT 44

INTRODUCTION

1. INTRODUCTION

1.1 ORGANIZATION PROFILE

1.2 ABOUT THE PROJECT

The **API Test Automation Framework for E-Commerce** is a production-ready, highly scalable testing solution engineered to validate the complex backend ecosystems of modern online retail platforms. Utilizing the **Karate Framework**, this project focuses on automating the end-to-end lifecycle of e-commerce transactions—including user authentication, product catalog exploration, shopping cart management, and order processing. The primary objective is to replace traditional, code-heavy automation methods with a streamlined, BDD-style (Behavior Driven Development) architecture that ensures API reliability, data integrity, and high-speed execution through parallel processing.

The framework is structured into specialized functional modules that reflect real-world business logic. The **Product Catalog and Shopping Cart modules** perform rigorous validation of state transitions and financial calculations, such as verifying that the discountedTotal matches the applied business rules. A core strength of the system is its **Data-Driven architecture**, which decouples test logic from data sources like JSON or CSV files, allowing a single test scenario to be executed against thousands of unique user data points. Furthermore, the system incorporates a **Performance and Reporting module** via Gatling integration, enabling developers to simulate high-concurrency traffic and monitor response times under heavy load.

Technically, the framework is built on a **Java-based Maven** infrastructure, leveraging the Gherkin syntax to create human-readable test scripts without the need for traditional step definitions. The environment is optimized for **Continuous Integration (CI/CD)**, supporting seamless execution through Jenkins or GitHub Actions. By utilizing **JUnit** for test orchestration and integrating mock servers for isolated development, the project provides a comprehensive environment for functional, performance, and regression testing. This ensures that every API endpoint—from JWT-based authentication to complex inventory checks—functions flawlessly before deployment to a production environment.

1.3 HARDWARE SPECIFICATION

System model : HP inspiron 13 laptop

System type : x64-based

Processor : Intel®core™ i5-8265U

RAM : 8GB RAM

1.4 SOFTWARE SPECIFICATION

Operating System : Windows, macOS, or Linux

Programming Language : Java

Build Tool : Apache Maven

Test Automation Framework : Karate Framework

Test Runner : JUnit

Performance Testing Tool : Gatling

1.5 SOFTWARE FEATURES

Programming Language : Java

Java is a high-level, object-oriented programming language widely used for developing secure, reliable, and scalable applications. It follows the principle of "Write Once, Run Anywhere" (WORA), enabling programs to run on any platform that supports the Java Virtual Machine (JVM). This platform independence makes Java suitable for enterprise, web, and distributed systems.

Java supports core object-oriented concepts such as encapsulation, inheritance, polymorphism, and abstraction, which promote modular design, code reusability, and maintainability. Its strong type-checking mechanism and automatic memory management through garbage collection enhance application stability and reduce runtime errors.

The language provides a rich set of built-in libraries and frameworks that simplify development tasks such as database connectivity, networking, multithreading, and security. Java is commonly used in building backend services, RESTful APIs, and large-scale enterprise applications using frameworks like Spring and Hibernate.

Overall, Java ensures high performance, robustness, and security, making it an ideal choice for developing efficient and maintainable server-side applications.

Build Tool : Apache Maven

Apache Maven is a powerful build automation and project management tool primarily used for Java-based applications. It simplifies the process of building, compiling, testing, and packaging software projects by following a standardized project structure and lifecycle. Maven uses a **Project Object Model (POM)** file to manage project configurations, dependencies, plugins, and build settings in a centralized and organized manner.

One of Maven's key features is **dependency management**, which automatically downloads and maintains required libraries from remote repositories. This eliminates manual handling of external JAR files and ensures consistent and reliable project builds across different environments. Maven also supports build lifecycle phases such as clean, compile, test, package, install, and deploy, enabling efficient automation of development and deployment tasks.

Additionally, Maven integrates seamlessly with popular IDEs, version control systems, and CI/CD pipelines, improving team collaboration and continuous integration processes. Its plugin architecture allows customization for tasks such as code testing, reporting, and packaging applications into JAR or WAR files.

Overall, Apache Maven enhances project organization, reduces build complexity, and ensures faster, reliable, and reproducible software development.

Test Automation Framework : Karate Framework

Karate Framework is an open-source test automation framework designed for API testing, performance testing, and test automation using a behavior-driven development (BDD) approach. Built on top of Cucumber and integrated with Java, it enables testers and developers to write test cases in a simple, readable format using **Gherkin syntax**, reducing the need for extensive programming knowledge.

Karate simplifies API testing by combining HTTP requests, assertions, and test logic into a single, easy-to-understand script. It supports testing of RESTful services, SOAP services, and web applications, allowing validation of request responses, status codes, headers, and JSON or XML data. Its built-in features eliminate the need for multiple external libraries, making automation faster and more efficient.

The framework also provides capabilities such as **data-driven testing, parallel test execution, mocking, and performance testing**, which improve test coverage and execution speed. Integration with tools like Maven, Jenkins, and CI/CD pipelines ensures continuous testing and faster feedback during development.

Performance Testing Tool : Gatling

Gatling is an open-source performance testing and load testing tool designed to evaluate the scalability, reliability, and responsiveness of web applications and APIs. It is built using Scala and provides a powerful, developer-friendly environment for simulating multiple users and analyzing system performance under various load conditions.

Gatling uses a **code-based testing approach**, allowing test scenarios to be written in a clear and expressive domain-specific language (DSL). This approach enables better flexibility, maintainability, and version control compared to traditional script-based tools. It supports testing of HTTP, RESTful APIs, WebSockets, and other protocols, making it suitable for modern web applications.

One of Gatling's key features is its ability to generate **detailed performance reports and visual analytics**, including response time, throughput, error rates, and system behavior under stress. These reports help identify bottlenecks, optimize performance, and ensure the application can handle real-world traffic. Gatling also supports parallel execution and seamless integration with Maven, Jenkins, and CI/CD pipelines for automated performance testing.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 PROBLEM DEFINITION

Modern e-commerce platforms rely heavily on complex backend APIs to handle critical business operations such as user authentication, product management, shopping cart updates, and order processing. Any failure or inconsistency in these APIs can lead to incorrect transactions, financial discrepancies, poor user experience, and potential revenue loss. Traditional manual testing and code-intensive automation approaches are often time-consuming, error-prone, and incapable of handling large-scale regression or high-concurrency scenarios efficiently.

This project addresses the challenges of ensuring API reliability, scalability, and performance through an automated testing framework. It replaces conventional testing methods with a Behavior Driven Development (BDD) based automation approach that enables faster development, easier maintenance, and improved readability of test cases. The framework validates business rules such as product availability, pricing accuracy, discount calculations, and transaction integrity to prevent logical and functional errors.

The system also incorporates data-driven testing to execute scenarios with multiple datasets, enabling comprehensive coverage across thousands of user flows. Performance testing is integrated to simulate real-world traffic conditions and detect bottlenecks under heavy loads. Continuous Integration support ensures that automated tests are executed regularly during deployment cycles, allowing early detection of defects.

By automating functional, regression, and performance validation of APIs, this project enhances system stability, reduces manual effort, and ensures secure and reliable operation of the e-commerce platform before production release.

2.2 SYSTEM STUDY

The existing testing approaches for e-commerce applications primarily depend on manual testing or traditional automation frameworks that require extensive programming effort and complex maintenance. As modern online retail platforms grow in size and complexity, backend APIs handle

critical operations such as authentication, product browsing, cart updates, payment processing, and order management. Manual validation of these APIs becomes time-consuming, error-prone, and inefficient.

Conventional automation tools often use code-heavy scripts that are difficult to read, update, and reuse. This increases maintenance costs and reduces productivity when APIs change frequently. Lack of standardized test structures and poor reusability of test data further slow down regression testing cycles. Additionally, many systems do not support parallel execution or data-driven testing, resulting in longer test execution times and limited coverage.

Performance testing is also often neglected or handled separately, making it difficult to evaluate system behavior under heavy user loads. Without proper load simulation and reporting mechanisms, issues such as slow response times, concurrency failures, and transaction breakdowns remain undetected until production deployment. The absence of continuous integration and automated pipelines further delays feedback and reduces software quality.

These limitations highlight the need for a scalable, maintainable, and automated API testing framework that ensures reliability, faster execution, improved coverage, and early detection of functional and performance defects.

DRAWBACKS OF THE EXISTING SYSTEM

- Heavy dependence on manual testing leads to slower release cycles.
- Code-intensive automation scripts are difficult to maintain and update.
- Limited support for data-driven testing reduces test coverage.
- Sequential execution increases overall test execution time.
- Lack of integrated performance testing fails to detect load-related issues.
- Poor reporting and monitoring make defect tracking difficult.
- Inadequate CI/CD integration delays continuous feedback and deployment.

2.3 PROPOSED SYSTEM

The proposedThe proposed system is an automated API Test Automation Framework designed to ensure the reliability, scalability, and performance of modern e-commerce backend services. The framework utilizes a Behavior Driven Development (BDD) approach to create readable and maintainable test scenarios that validate critical business operations such as authentication, product management, cart functionality, and order processing.

The system automates functional, regression, and integration testing by executing structured test cases against backend APIs using reusable test scripts. It supports data-driven testing to validate multiple input combinations and diverse user flows, ensuring broader coverage of real-world scenarios. Performance testing capabilities are incorporated to simulate concurrent users and heavy traffic loads, enabling early detection of bottlenecks and system limitations.

Continuous Integration (CI) pipelines are integrated to automatically execute tests during every build and deployment cycle, ensuring quick identification of defects. The framework also generates detailed reports and logs to provide insights into test results, failures, and overall system health. By combining automation, scalability, and performance validation, the proposed system delivers a reliable and production-ready testing solution for e-commerce platforms.

2.3.1 NEED FOR THE PROPOSED SYSTEM

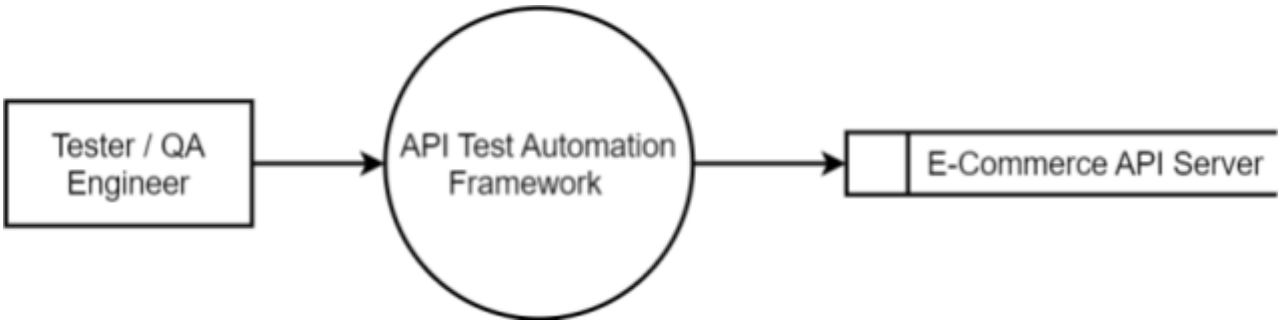
- To eliminate time-consuming and error-prone manual testing processes.
- To ensure reliable validation of critical e-commerce APIs before production deployment.
- To provide faster regression testing during frequent feature updates and releases.
- To improve test coverage using automated and data-driven testing techniques.
- To detect performance issues and bottlenecks under high user traffic conditions.
- To enable continuous testing through CI/CD integration for early defect detection.
- To reduce maintenance effort using readable BDD-based test cases.
- To ensure secure, stable, and consistent API behavior for better user experience and business reliability.

SYSTEM DESIGN

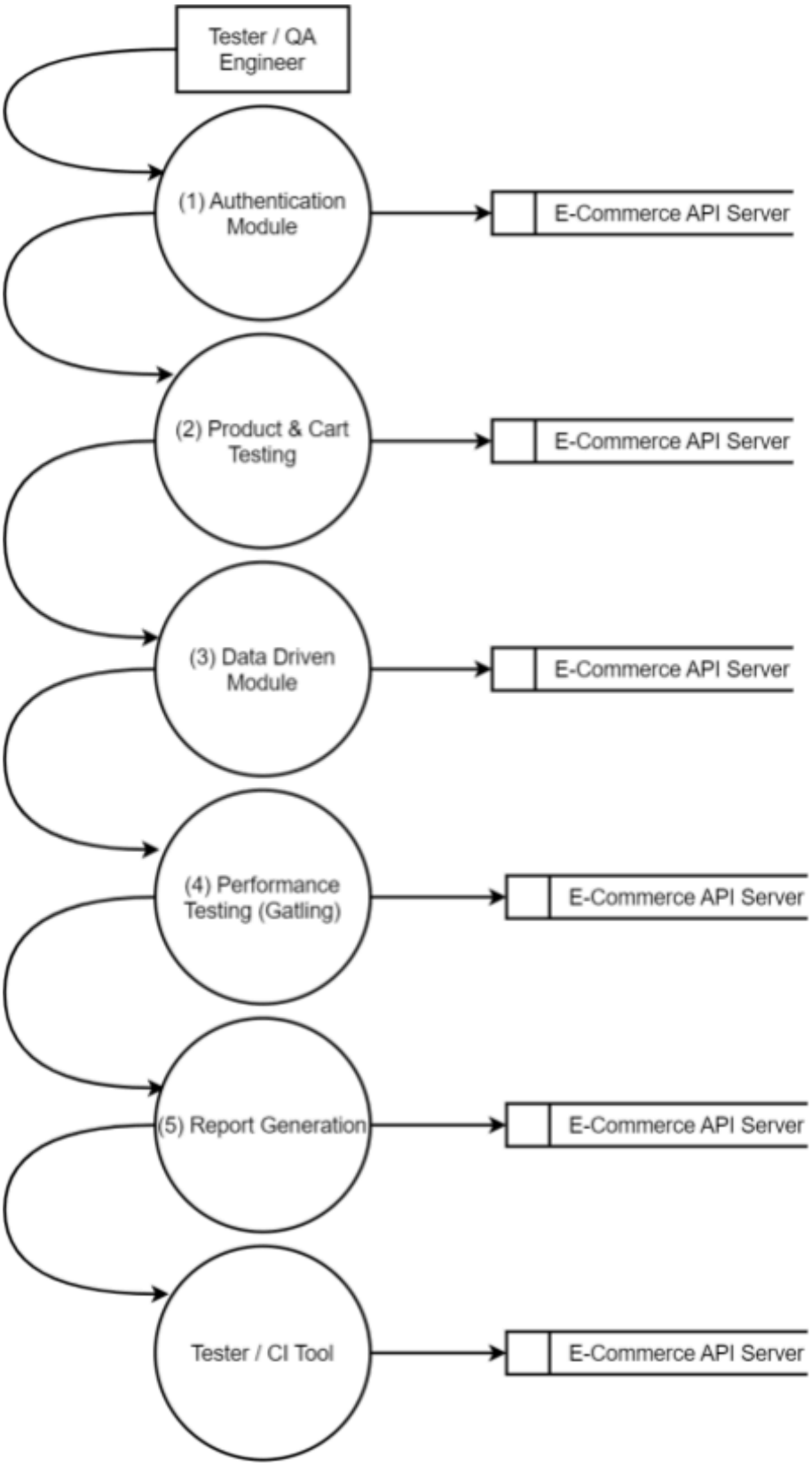
3. SYSTEM DESIGN

3.1 DATA FLOW DIAGRAM

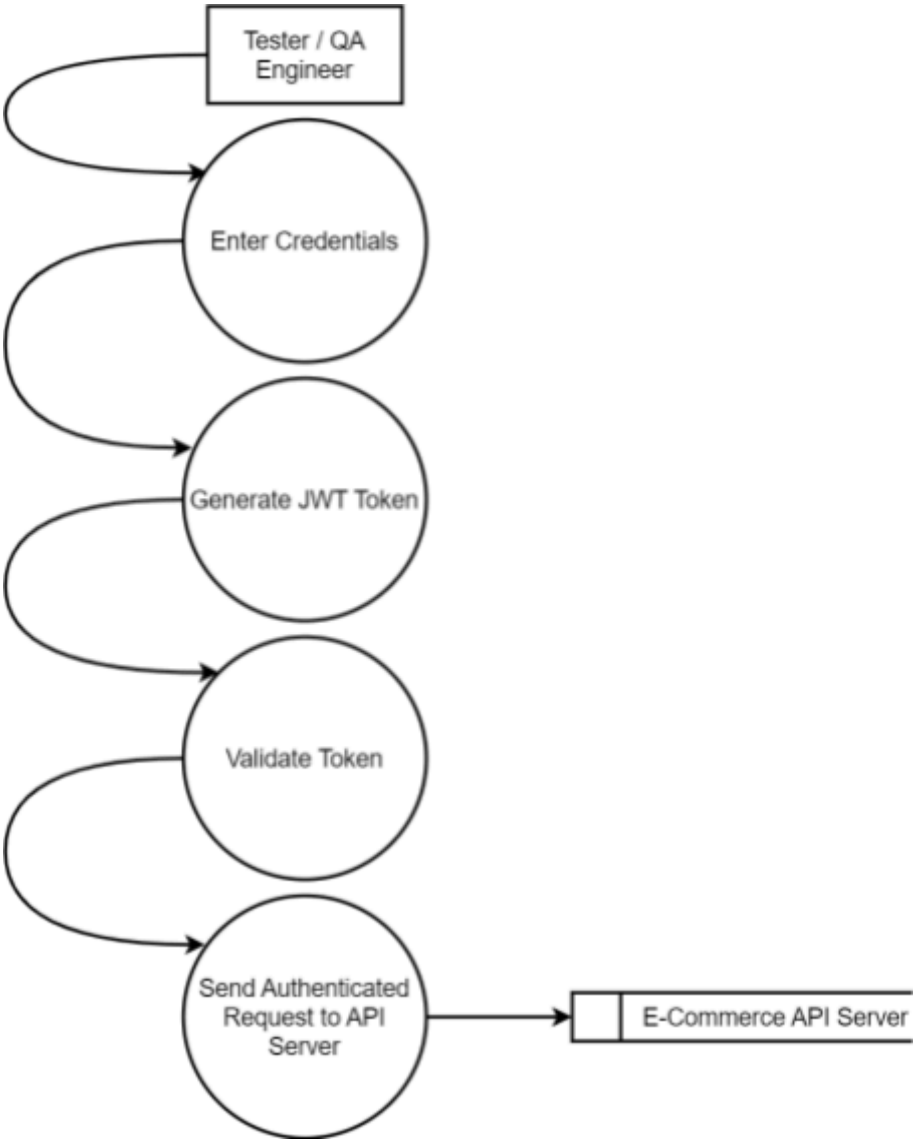
LEVEL 0



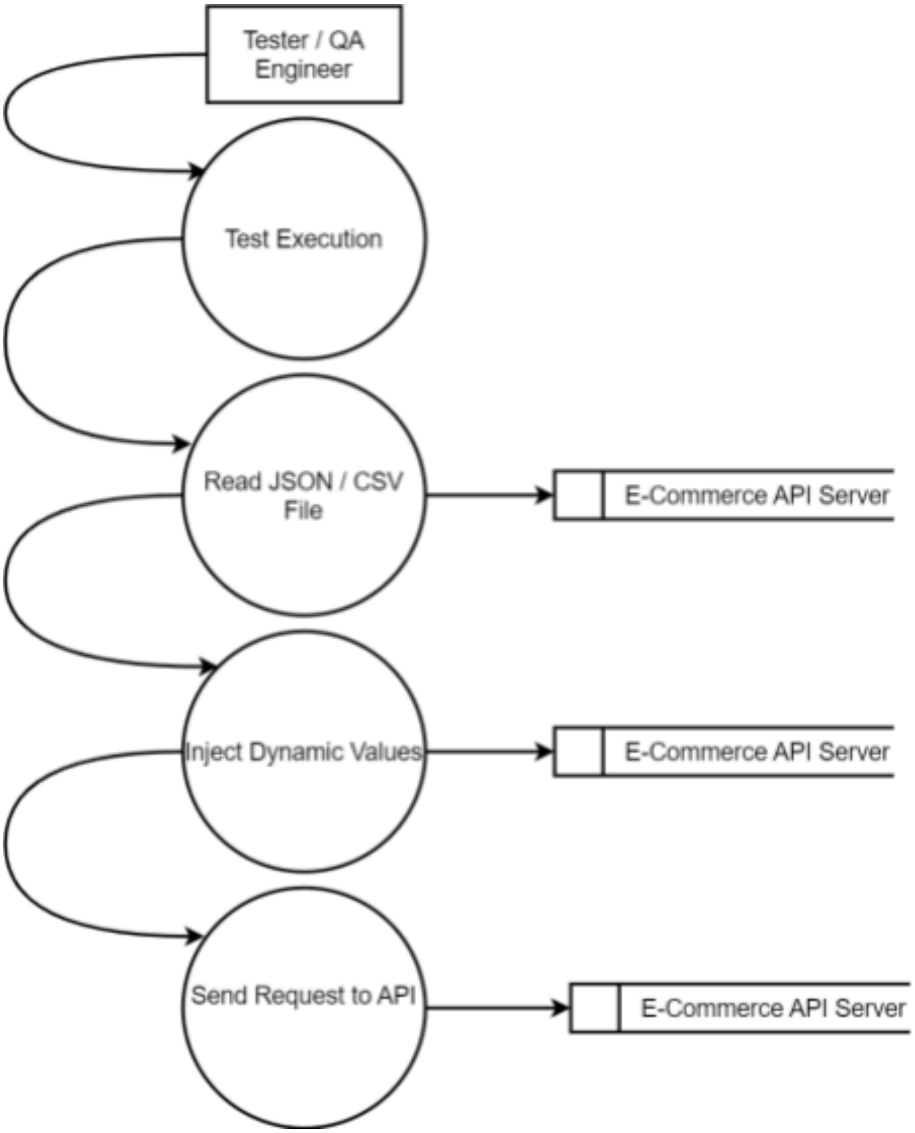
LEVEL 1


Tester / QA Engineer
(1) Authentication Module
E-Commerce API Server
(2) Product & Cart Testing
E-Commerce API Server
(3) Data Driven Module
E-Commerce API Server
(4) Performance Testing (Gatling)
E-Commerce API Server
(5) Report Generation
E-Commerce API Server
Tester / CI Tool
E-Commerce API Server

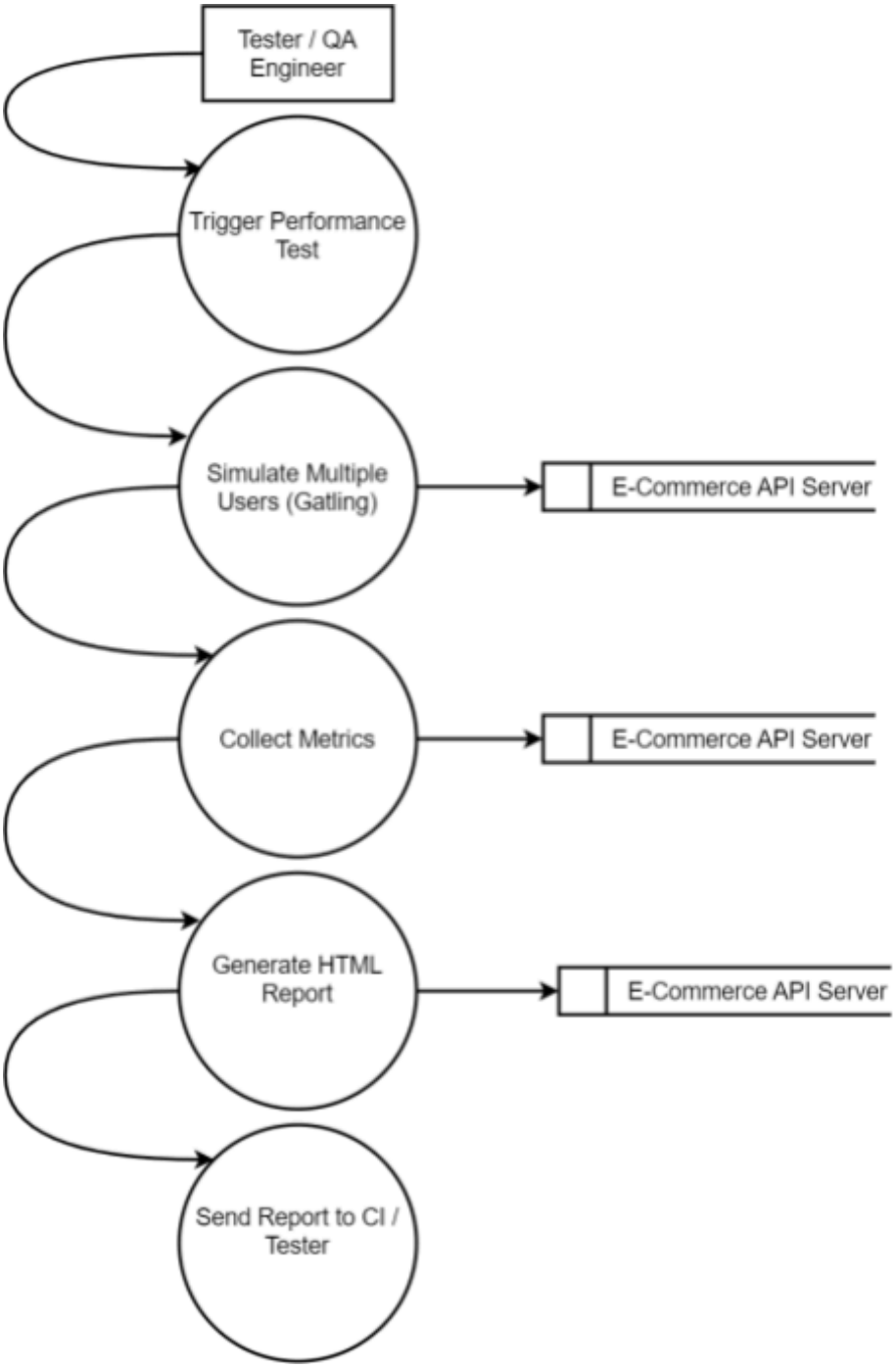
LEVEL 2 : Authentication testing.



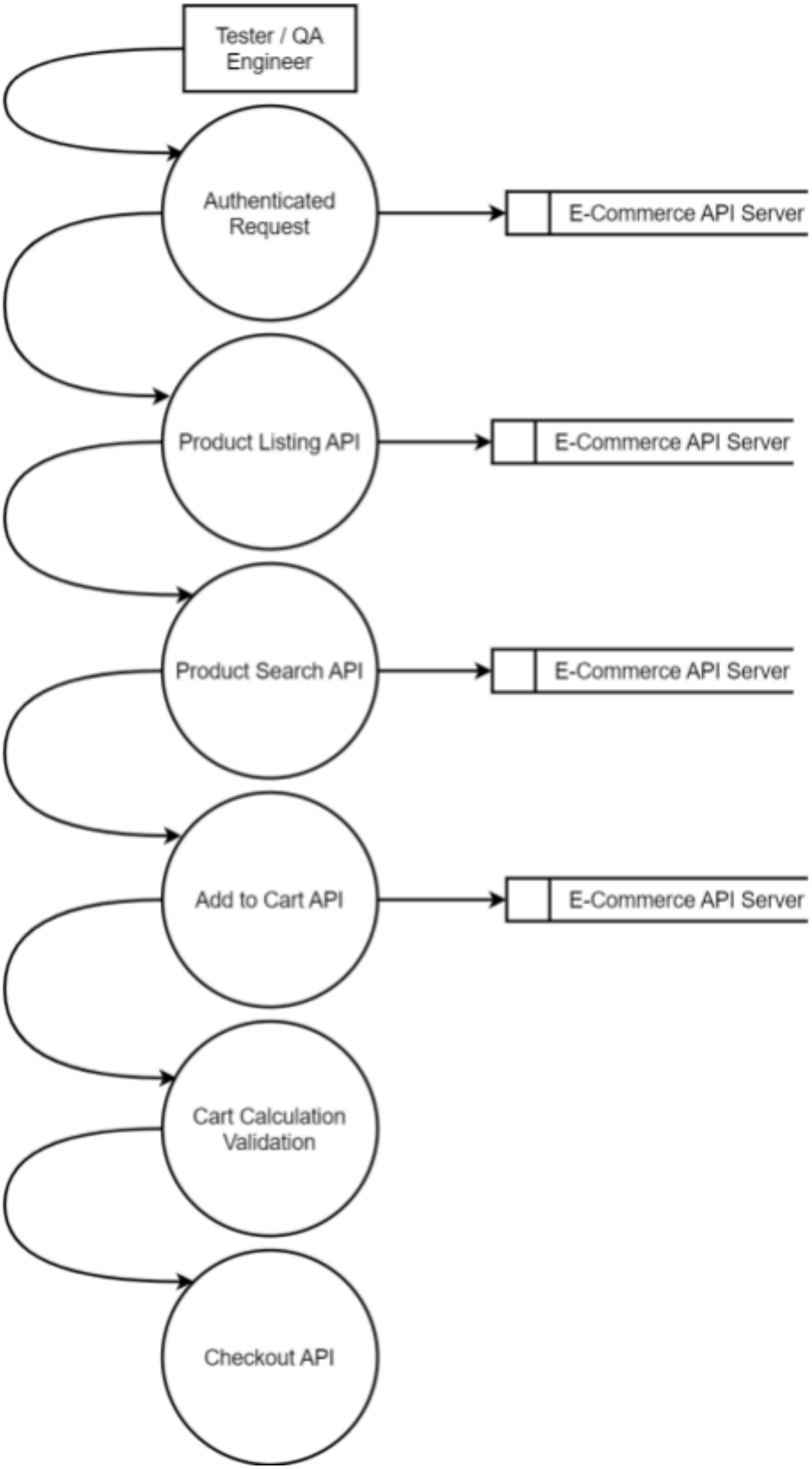
LEVEL 2 : Data driven module



LEVEL 2 : Performance & Reporting Module



Level 2 : Product Cart Module



3.2 ERD

3.3 FILE SPECIFICATION

Table Name: USER_MASTER
Purpose: Stores tester and admin user information who execute and manage the automation framework.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
UserId	INT	—	Primary Key, Identity	Unique user id
UserName	VARCHAR	100	NOT NULL	Name of the user
Email	VARCHAR	150	UNIQUE	Email address
Role	VARCHAR	50	NOT NULL	Admin/Tester
PasswordHash	VARCHAR	255	NOT NULL	Encrypted password
CreatedAt	DATETIME	—	DEFAULT	Account creation time

Table Name: PROJECT_MASTER
Purpose: Stores automation project details.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION

ProjectId	INT	—	Primary Key, Identity	Unique project id
ProjectName	VARCHAR	150	NOT NULL	Name of project
Description	VARCHAR	500	NULL	Project details
CreatedBy	INT	—	Foreign Key	Created user
CreatedDate	DATETIME	—	—	Creation date

Table Name: TEST_CASE_MASTER
Purpose: Stores all API test cases written using Karate framework.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
TestCaseId	INT	—	Primary Key	Unique test case id
ProjectId	INT	—	Foreign Key	Linked project
TestName	VARCHAR	200	NOT NULL	Test scenario name
ModuleName	VARCHAR	100	—	Auth/Cart/Product
FeatureFilePath	VARCHAR	300	NOT NULL	Karate feature file path
Priority	VARCHAR	20	—	High/Medium/Low
CreatedDate	DATETIME	—	—	Created time

Table Name: TEST_DATA_SET
Purpose: Stores external data used for data-driven testing (JSON/CSV files).

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
DataSetId	INT	—	Primary Key	Dataset id

TestCaseId	INT	—	Foreign Key	Linked test case
FileName	VARCHAR	255	NOT NULL	Data file name
FileType	VARCHAR	20	—	JSON/CSV
RecordCount	INT	—	—	Number of records
UploadedDate	DATETIME	—	—	Upload date

Table Name: TEST_EXECUTION
Purpose: Stores each test execution run details.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
ExecutionId	INT	—	Primary Key	Execution id
ProjectId	INT	—	Foreign Key	Project reference
StartTime	DATETIME	—	NOT NULL	Execution start time
EndTime	DATETIME	—	—	Execution end time
Environment	VARCHAR	50	—	Dev/QA/Prod
TriggeredBy	INT	—	Foreign Key	Executed by user
Status	VARCHAR	20	—	Passed/Failed

Table Name: TEST_RESULT
Purpose: Stores individual test case results for every execution.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
ResultId	INT	—	Primary Key	Result id

ExecutionId	INT	—	Foreign Key	Linked execution
TestCaseId	INT	—	Foreign Key	Linked test case
Status	VARCHAR	20	NOT NULL	Pass/Fail
ResponseTimeMs	INT	—	—	API response time
ErrorMessage	VARCHAR	500	NULL	Failure details

Table Name: PERFORMANCE_METRICS
Purpose: Stores Gatling performance testing metrics.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
MetricId	INT	—	Primary Key	Metric id
ExecutionId	INT	—	Foreign Key	Linked execution
AvgResponseTime	INT	—	—	Average response time
MaxResponseTime	INT	—	—	Maximum response time
Throughput	INT	—	—	Requests per second
ErrorRate	DECIMAL	5,2	—	Failure percentage

Table Name: ENVIRONMENT_CONFIG
Purpose: Stores API environment configuration details.

FIELDNAME	DATATYPE	SIZE	CONSTRAINTS	DESCRIPTION
EnvId	INT	—	Primary Key	Environment id
EnvName	VARCHAR	50	UNIQUE	Dev/QA/Prod

BaseURL	VARCHAR	300	NOT NULL	API base URL
AuthType	VARCHAR	50	—	JWT/OAuth
CreatedDate	DATETIME	—	—	Created date

3.4 MODULE SPECIFICATION

The system is divided into the following core modules:

- **AUTHENTICATION MODULE**
- **PRODUCT CATALOG MODULE**
- **SHOPPING CART MODULE**
- **DATA-DRIVEN MODULE**
- **PERFORMANCE & REPORTING MODULE**

AUTHENTICATION MODULE

The Authentication Module manages secure access to the system and validates user login functionalities. It ensures that only authorized users can interact with the e-commerce APIs.

User Authentication

This sub-module is used to verify user credentials and generate secure tokens such as JWT for accessing protected endpoints.

Token Validation

This sub-module validates authentication tokens for each API request and ensures secure communication between client and server.

PRODUCT CATALOG MODULE

The Product Catalog Module handles the validation of all product-related operations provided by the e-commerce backend services.

Product Listing

This sub-module is used to verify that all available products are retrieved correctly from the API.

Product Search and Filter

This sub-module is used to validate searching and filtering of products based on category, price, and rating.

Product Data Validation

This sub-module checks fields such as id, title, description, category, price, rating, and stock to ensure that the returned product structure matches the expected format.

SHOPPING CART MODULE

The Shopping Cart Module manages cart-related operations and validates complex business logic such as totals and discounts.

Cart Creation

This sub-module is used to create new shopping carts for users.

Add/Update Products

This sub-module allows adding, updating, and removing products in the cart.

Cart Calculation

This sub-module verifies calculated fields such as total, discountedTotal, and totalQuantity to ensure they match the business rules.

Cart Status Validation

This sub-module ensures that cart states are properly updated after each operation.

DATA-DRIVEN MODULE

The Data-Driven Module separates test logic from test data, enabling the framework to execute multiple test scenarios using different input values.

External Data Management

This sub-module handles JSON, CSV, and Excel files for storing test data.

Dynamic Variable Injection

This sub-module dynamically injects values such as userId, productId, and quantity into API requests.

Environment Configuration

This sub-module manages base URLs, headers, and environment settings through configuration scripts.

PERFORMANCE & REPORTING MODULE

The Performance and Reporting Module monitors system behavior under load and provides detailed execution reports.

Performance Testing

This sub-module integrates with Gatling to simulate high-concurrency users and measure API response times.

Execution Monitoring

This sub-module captures request logs, status codes, timestamps, and error rates.

Report Generation

This sub-module generates HTML and JUnit XML reports for analysis and CI/CD integration.

TESTING AND IMPLEMENTATION

4. TESTING AND IMPLEMENTATION

4.1 SYSTEM TESTING

Got it — using your **college project report format**, here is the **TESTING AND IMPLEMENTATION** section written specifically for your **API Test Automation Framework for E-Commerce (Karate + Maven + Gatling)** project.

You can paste this directly into **Chapter 4** of your report.

TESTING AND IMPLEMENTATION

4.1 SYSTEM TESTING

System testing ensures that the complete **API Test Automation Framework for E-Commerce** satisfies all functional and non-functional requirements. The testing process validates that each API endpoint, test scenario, and automation module performs accurately and reliably under different conditions.

The framework undergoes multiple testing techniques such as **Unit Testing, Integration Testing, System Testing, User Acceptance Testing, and Performance & Security Testing**.

Unit Testing

Focus: Individual test scripts and functions

Methodology:
Unit testing verifies each automation component independently. Every feature file, reusable function, and utility method is tested in isolation to ensure correct behavior. Test-Driven Development (TDD) principles are followed where possible.

Tools:
JUnit, Karate built-in assertions, Maven test lifecycle

Examples:

- Testing authentication API for valid JWT token generation.
- Testing product catalog API for correct product retrieval and filtering.

- Testing cart calculation logic for correct discountedTotal and price validation.
- Testing reusable helper functions for request/response validation.

Integration Testing

Focus: Interaction between different modules

Methodology:
Integration testing verifies smooth communication between modules such as authentication, catalog, cart, and order processing. It ensures correct data flow across services.

Examples:

- Testing login → product selection → add to cart → checkout workflow.
- Testing order creation after successful payment confirmation.
- Testing integration between Karate scripts and mock servers.
- Testing CI/CD execution through Jenkins or GitHub Actions pipelines.

System Testing

Focus: End-to-end testing of the entire framework

Methodology:
System testing validates the complete framework as a whole by simulating real-world e-commerce scenarios. It ensures that all APIs work together reliably.

Examples:

- Running full regression suites covering all business flows.
- Executing multiple user scenarios in parallel.
- Verifying API response times and correctness.
- Checking reporting dashboards and log generation.

User Acceptance Testing (UAT)

Focus: Validate usability and effectiveness from the tester's perspective

Methodology:
Test engineers and stakeholders execute test scenarios and review outputs to ensure the framework is easy to use, readable, and maintainable.

Examples:

- Writing new test cases using Gherkin syntax without coding.
- Validating reports for clarity and correctness.
- Ensuring test scripts are understandable for non-technical stakeholders.
- Confirming easy execution through Maven commands.

Performance and Security Testing
Performance Testing

Focus: System behavior under heavy load

Methodology:
Gatling is used to simulate concurrent users and measure API performance metrics such as response time, throughput, and failure rates.

Examples:

- Simulating thousands of users accessing the product catalog simultaneously.
- Measuring checkout response time during peak load.
- Identifying bottlenecks in order processing APIs.

Security Testing

Focus: Identify vulnerabilities and ensure secure APIs

Methodology:
Validation of authentication, authorization, and input handling to prevent attacks.

Examples:

- Testing invalid tokens and unauthorized access.
- Verifying secure header implementation.
- Checking for injection and malformed input handling.
- Ensuring sensitive data is not exposed in responses.

4.2 SYSTEM IMPLEMENTATION

System implementation involves deploying the tested **API Test Automation Framework** into a production-ready environment. The implementation is carried out in phases to ensure stability and smooth operation.

Phase 1: Development and Unit Testing

In this phase, Karate feature files, reusable test modules, and supporting utilities are developed. Unit tests are written for all functions and executed using Maven. Continuous testing ensures that each module works independently.

Phase 2: Integration Testing

All modules are combined and tested together. API workflows such as authentication, cart management, and order processing are validated. Integration with mock servers and databases is verified.

Phase 3: System Testing and User Acceptance Testing (UAT)

The framework is deployed in a staging environment for full regression testing. End users execute test suites and provide feedback regarding script readability, reporting, and usability. Necessary improvements are implemented.

Phase 4: Deployment and Go-Live

The framework is deployed in the CI/CD pipeline using Jenkins or GitHub Actions. Maven build scripts automate test execution. Gatling performance tests are scheduled for periodic monitoring. Final checks ensure stable and reliable execution.

Phase 5: Post-Implementation Support and Maintenance

After deployment, continuous monitoring and maintenance are carried out. Test cases are updated as APIs evolve. Logs, reports, and performance metrics are reviewed regularly. New features and enhancements are incorporated to improve coverage and reliability.

CONCLUSION AND SUGGESTION

5. CONCLUSION AND SUGGESTION

5.1 CONCLUSION

The proposed API Test Automation Framework provides a reliable, scalable, and efficient solution for validating modern e-commerce backend services. By adopting a Behavior Driven Development (BDD) approach, the framework enables the creation of readable and maintainable test scenarios that clearly represent business requirements. The automation of functional, regression, and integration testing reduces manual effort, minimizes human errors, and accelerates the overall testing process.

The integration of data-driven testing ensures broader test coverage across multiple user flows, while performance testing helps identify system bottlenecks under heavy traffic conditions. Continuous Integration (CI) pipelines further enhance reliability by executing tests automatically during every build and deployment cycle, enabling early defect detection. Overall, the framework improves software quality, ensures stable API behavior, and supports faster and more secure releases for e-commerce platforms.

5.2 SUGGESTION

- **Automated Test Scheduling**: Configure scheduled test runs during off-peak hours to continuously monitor API stability.

- **Enhanced Security Testing**: Integrate security testing tools such as OWASP ZAP to detect vulnerabilities like injection attacks and authentication flaws.

- **Parallel Execution**: Implement parallel test execution to reduce overall testing time and improve efficiency.

- **Improved Reporting**: Add advanced dashboards and visual reports to track trends, failures, and performance metrics more effectively.

- **Cloud-Based Scalability**: Deploy tests on cloud infrastructure to simulate large-scale concurrent users and real-world traffic conditions.

- **Test Data Management**: Use automated test data generation and cleanup strategies to maintain consistency across environments.

- **Future Enhancements**: Incorporate AI-driven test case generation and self-healing scripts to automatically adapt to API changes.

BIBLIOGRAPHY

BIBLIOGRAPHY

I. BOOK REFERENCES

1. Freeman, A. *Pro ASP.NET Core MVC 2*.
   Apress, Seventh Edition, 2018.

2. Esposito, D. *Modern Web Development with ASP.NET Core MVC*.
   Microsoft Press, First Edition, 2018.

3. Banks, A., & Porcello, E. *Learning React: Modern Patterns for Developing React Apps*.
   O'Reilly Media, Second Edition, 2020.

4. Duckett, J. *HTML and CSS: Design and Build Websites*.
   Wiley, First Edition, 2011.

5. Duckett, J. *JavaScript and JQuery: Interactive Front-End Web Development*.
   Wiley, First Edition, 2014.

6. Douglas, K., & Douglas, S. *PostgreSQL: Up and Running*.
   O'Reilly Media, Third Edition, 2017.

7. Delaney, K. *SQL Server Internals*.
   Microsoft Press, Second Edition, 2016.

II. URL REFERENCES

1. 1.Microsoft .NET Documentation (ASP.NET Core Web API)
   https://learn.microsoft.com/dotnet
2. React Official Documentation
   https://react.dev
3. Bootstrap Official Website
   https://getbootstrap.com
4. PostgreSQL Official Documentation
   https://www.postgresql.org/docs/
5. Microsoft SQL Server Documentation
   https://learn.microsoft.com/sql

6. SonarQube Official Documentation
   https://docs.sonarsource.com
7. OWASP ZAP Official Documentation
   https://www.zaproxy.org/docs/
8. OWASP Top 10 – Web Application Security Risks
   https://owasp.org

APPENDIX - A (FORMS)

APPENDIX - A (FORMS)

A1. LOGIN PAGE

A2. HOME PAGE

A3. USER ENTRY DETAILS

APPENDIX - B (REPORTS)

APPENDIX - B (REPORTS)

B1. USER REPORT